

DECODELABS DATA SCIENCE INTERNSHIP

Project 2 — Fraud Detection Pipeline

Imbalanced Classification with SMOTE, Logistic Regression & Random Forest

PROGRAM Batch 2026	ROLE Data Science Intern	AUTHOR Muhammad Usaid
------------------------------	------------------------------------	---------------------------------

Internship Project Documentation

1. Objective & Project Context

This project builds a **leak-free supervised classification pipeline** that flags potentially fraudulent/problem orders using a proxy target. It compares a linear baseline (**Logistic Regression**) with an ensemble model (**Random Forest**), with both models tuned using **GridSearchCV** to optimize recall.

The project builds directly on **Project 1**, using its cleaned dataset as the starting point. Accuracy is deliberately excluded from evaluation.

Important: the dataset does not contain a genuine fraud/legitimate label. The target is a proxy for problem orders and must not be interpreted as confirmed financial fraud.

Proxy target definition

IsFraud = 1 if **OrderStatus** is **Returned** or **Cancelled**; otherwise IsFraud = 0.

This distinction matters because returned or cancelled orders can correlate with disputes, suspicious activity, or operational problems, but they are **not automatically fraudulent**. The machine-learning architecture nevertheless follows principles appropriate for a real fraud-detection workflow.

Project Requirements → Implementation

Requirement	Implementation
Classification target	IsFraud proxy created in src/target_builder.py
Class imbalance	SMOTE inside imblearn.pipeline.Pipeline
Multiple algorithms	Logistic Regression and Random Forest
Cross-validation	GridSearchCV
Optimization	scoring="recall"
Leakage prevention	Stratified split before resampling and scaling
Evaluation	Precision, Recall, F1-Score, ROC-AUC
Model analysis	Confusion matrices, ROC/PR curves, feature importance
Testing	17 automated tests

2. Leak-Free Pipeline

Trap #1 — The Illusion of Accuracy

Accuracy is not calculated or reported anywhere in the evaluation system. The project focuses on Precision, Recall, F1-Score and ROC-AUC. This is appropriate for fraud-detection-style problems where missing a potentially problematic transaction can be more costly than generating a false alarm.

Trap #2 — Data Leakage

SMOTE and feature scaling are applied inside the machine-learning pipeline rather than before the train/test split. Synthetic samples are generated only from the training portion of each cross-validation fold.

Original Dataset ↓ Build IsFraud Proxy Target ↓ Feature Engineering ↓ Remove Leaky Features ↓ Stratified Train/Test Split ↓ ████████████████████ Test Set → Final Evaluation ■ █████ Training Set ↓ Cross Validation ↓ SMOTE ↓ Scaling* ↓ Model * Scaling is used for Logistic Regression but not for Random Forest.

Because SMOTE is contained inside `imblearn.pipeline.Pipeline`, the held-out validation/test data is never resampled. This is the key safety property of the project.

3. Target, Dataset & Class Balance

The input is `data/raw/project1_cleaned_dataset.csv`, the cleaned and feature-engineered output from Project 1.

Dataset characteristic	Value
Orders	1,200
Original columns	25
Missing values	None
Modeling features before split	37
Train / Test	960 / 240

Label	Count	Percentage
Legitimate (0)	703	58.58%
Fraud-proxy (1)	497	41.42%
Total	1,200	100%

Class-imbalance context: 41.42% is not extremely imbalanced compared with real-world fraud datasets, where fraud can be below 1%. SMOTE is still implemented to demonstrate the correct technique and the recall-first workflow.

4. Feature Engineering & Leakage Prevention

Feature	Formula / Source	Purpose
AvgItemValue	TotalPrice / ItemsInCart	Average value per cart item
ItemsPerOrder	ItemsInCart / (Quantity + 1)	Cart-to-purchase relationship
IsHighValue	TotalPrice > 95th percentile	Unusually high-value orders
PricePerUnit	TotalPrice / Quantity	Order value normalized by quantity
HasDiscount	CouponUsed	Coupon usage indicator
WeekendFraud	IsWeekendOrder × IsFraud	Exploratory analysis only

Explicitly removed before modeling

- **OrderStatus** — it is the source used to construct IsFraud. Keeping it would expose the answer to the model.
- **WeekendFraud** — it directly contains the target and is therefore exploratory only.
- **OrderID, CustomerID, TrackingNumber, ShippingAddress** — identifiers that do not provide meaningful generalizable predictive information.
- Other raw/superseded columns are removed where their information is already represented by engineered features.

Stratified split: 80% training (960 rows) and 20% testing (240 rows). The split happens before SMOTE or scaling, keeping the test set untouched and representative of the original distribution.

5. Modeling & Hyperparameter Tuning

Both algorithms are tuned using **GridSearchCV**. The optimization metric is **Recall**, rather than accuracy, because the project prioritizes identifying as many potentially problematic orders as possible.

5.1 Logistic Regression Pipeline

StandardScaler → SMOTE → Logistic Regression

Selected setting	Value
C	0.1
solver	liblinear
SMOTE k_neighbors	7
Best cross-validation recall	0.507

Scaling is used because Logistic Regression is sensitive to feature magnitude and regularization.

5.2 Random Forest Pipeline

SMOTE → Random Forest

Selected setting	Value
class_weight	balanced
max_depth	None
n_estimators	200
SMOTE k_neighbors	5
Best cross-validation recall	0.274

A scaler is not required for Random Forest because tree-based models are generally insensitive to feature scale.

Model selection rule: recall is the primary criterion; accuracy is intentionally excluded.

6. Final Test-Set Results

Metric	Logistic Regression	Random Forest
Precision	0.377	0.344
Recall	0.434	0.222
F1-Score	0.404	0.270
ROC-AUC	0.437	0.442

0.434	0.404	0.442
Best recall · Logistic Regression	Best F1 · Logistic Regression	Best ROC-AUC · Random Forest

Best model by recall: Logistic Regression

Logistic Regression is selected as the best model according to the project's primary selection criterion: **Recall = 0.434**. Random Forest has a slightly higher ROC-AUC (0.442 vs 0.437), but recall remains the stated primary criterion.

Results interpretation

Neither model demonstrates strong predictive performance on the test set. Both ROC-AUC scores are below 0.50, suggesting that the available features contain very limited predictive signal for the IsFraud proxy target.

This is not necessarily a pipeline failure. Since OrderStatus is the source of the target, it must be removed to prevent leakage. A correctly implemented model cannot manufacture meaningful predictive signal when the available features do not contain it.

7. Feature Importance & Generated Reports

The README identifies the following as the most important Random Forest features:

- **TotalPrice**
- **PricePerUnit**
- **AvgItemValue**
- **UnitPrice**
- **OrderMonth**

Feature importance should be interpreted cautiously because the overall ROC-AUC indicates that these features provide limited predictive power for the proxy target.

Generated outputs

Output	Purpose
data/processed/orders_with_fraud_features.csv	Processed modeling dataset
reports/class_balance.csv	Class distribution
reports/model_evaluation_metrics.csv	Final evaluation metrics
reports/logistic_regression_best_params.csv	Best Logistic Regression settings
reports/random_forest_best_params.csv	Best Random Forest settings
reports/random_forest_feature_importance.csv	Feature-importance ranking
reports/figures/00_class_balance.png	Class-balance figure
reports/figures/01_fraud_detection_dashboard.png	Evaluation dashboard

The dashboard contains confusion matrices, ROC curves, Precision-Recall curves, model metric comparison, and Random Forest feature importance.

8. Testing, Repository & Quick Start

Testing

The project contains **17 automated tests**. The test suite validates that leakage-prone columns are removed, the final feature matrix is numeric, splitting happens before resampling, SMOTE does not modify the held-out test set, stratification preserves class distribution, the modeling pipeline can be fitted, and **EvaluationResult does not contain an accuracy field**.

Repository structure

```
. ├── data/ ─┬── raw/project1_cleaned_dataset.csv ─┬── processed/orders_with_fraud_features.csv ─┬──
notebooks/Fraud_Detection_Pipeline.ipynb ─┬── reports/ ─┬── class_balance.csv ─┬──
model_evaluation_metrics.csv ─┬── logistic_regression_best_params.csv ─┬── random_forest_best_params.csv
├── random_forest_feature_importance.csv ─┬── figures/ ─┬── 00_class_balance.png ─┬──
01_fraud_detection_dashboard.png ─┬── src/ ─┬── data_loader.py ─┬── target_builder.py ─┬──
feature_engineering.py ─┬── modeling.py ─┬── evaluation.py ─┬── plotting.py ─┬── pipeline.py ─┬──
tests/test_pipeline.py ─┬── main.py ─┬── requirements.txt ─┬── LICENSE ─┬── README.md
```

Quick start

```
python -m venv venv # Windows PowerShell venv\Scripts\activate # Linux / macOS source venv/bin/activate pip
install -r requirements.txt python main.py pytest -v jupyter notebook
notebooks/Fraud_Detection_Pipeline.ipynb
```


9. Tech Stack, Key Lessons & Conclusion

Area	Implementation
Programming	Python 3.12
Data Processing	pandas, NumPy
Machine Learning	scikit-learn, Logistic Regression, Random Forest, GridSearchCV
Imbalanced Learning	imbalanced-learn, SMOTE, imblearn.pipeline.Pipeline
Evaluation	Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix, ROC/PR Curves
Visualization	Matplotlib, Seaborn
Testing	pytest
Development	Jupyter Notebook, nbconvert, VS Code, Git, GitHub

Key lessons

- **Accuracy is not always the right metric.** Fraud-detection-style problems should emphasize Precision, Recall, F1 and ROC-AUC.
- **Data leakage can create misleading results.** Target-derived features such as OrderStatus must not be passed to the model.
- **SMOTE must be used correctly.** It belongs inside the training pipeline so validation/test data remains untouched.
- **Good pipelines can still produce poor scores.** Weak scores can be an honest reflection of weak predictive signal rather than an implementation failure.

Author's note

The IsFraud label is derived from Returned + Cancelled orders and should therefore be interpreted only as a **problem-order / fraud-proxy classification task**. The near-random ROC-AUC results are reported as produced by the pipeline. No accuracy score has been added to make the results appear better, and no leaky features have been retained to artificially increase model performance.

Conclusion

The project demonstrates a reproducible, leak-free, recall-focused classification workflow. Based on the documented test results, **Logistic Regression with SMOTE** is the selected model because it achieves the strongest recall and the best overall reported metrics except ROC-AUC, where Random Forest is marginally higher.

Related Project

Project 1 — Advanced EDA & Feature Engineering → Cleaned Dataset → Project 2 — Fraud Detection Pipeline

License: MIT License

Built as part of the DecodeLabs Data Science Industrial Training Program — Batch 2026.